

Tarea 4: Introducción a la computación de alto rendimiento (6%)

Marlon Brenes*

El propósito de esta tarea es practicar los conceptos de paralelismo de memoria compartida y distribuida. La tarea involucra entregar código fuente de C++ **con su respectivo reporte detallando el análisis**. Usted debe entregar los siguientes archivos para ser evaluados:

- `fourier.cpp`
- `dot_product.cpp`
- `reporte4.pdf/reporte4.doc/reporte4.docx/reporte4.ipynb` (dependiendo de cual formato escoja para su reporte)

PARTE I: MEMORIA COMPARTIDA (3%)

La transformada de Fourier discreta (DFT) es una de las operaciones con mayor rango de aplicabilidad en la ciencia moderna. Recordemos que evaluar la transformada de Fourier equivale a evaluar los N *coeficientes de Fourier*, i.e.,

$$c_k = \sum_{n=0}^{N-1} y_n \exp\left(-i \frac{2\pi kn}{N}\right), \quad (1)$$

donde c_k es el k -ésimo coeficiente de Fourier. Recuerde que $y_n = f(x_k)$ es la función de la cual queremos obtener la transformada de Fourier evaluada en el k -ésimo punto de colocación.

Se le ha entregado una implementación de la DFT usando C++. En esta implementación, no se utilizan las simetrías de conjugación vistas en clase ni el algoritmo *divide-and-conquer* que reduce la complejidad del cálculo. Es una implementación directa de la transformada. Note que el cálculo usa $N = 8192$ elementos para la transformada. Dicho cálculo debería tomar algunos segundos en una computadora personal.

- Diseñe una estrategia de paralelización para evaluar la DFT y describa dicha estrategia en su reporte. Piense en los accesos de memoria concurrentes y como evitar condiciones de carrera (*race conditions*). Dé una explicación en su reporte. (0.5%)
- Implemente la paralelización del algoritmo en el archivo `fourier.cpp`. (1.0%)
- Evalúe la aceleración (*speed-up*) usando la cantidad de hilos/núcleos físicos que tenga disponibles. (0.5%)
- Realice un gráfico de aceleración vs hilos. (1.0%)

PARTE II: MEMORIA DISTRIBUIDA (3.0%)

Su tarea es implementar el producto interno de dos vectores de tamaño N utilizando paralelismo de memoria distribuida. La operación corresponde a:

$$k = \mathbf{a}^T \cdot \mathbf{b} = [a_1 \ a_2 \ \cdots \ a_N] \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{bmatrix} \quad (2)$$

Para implementar la operación, siga las siguientes instrucciones:

- Inicialice el ambiente de MPI como vimos en clase (0.25%)

* marlon.brenes@utoronto.ca

- Declare 4 vectores usando `std::vector` como arreglos de datos: `a`, `b`, `local_a` y `local_b` (0.25%)
- En el proceso 0, utilice el método `resize()` para asignar suficiente memoria para N elementos para los vectores `a` y `b` (0.25%)
- Para todos los procesos, utilice el método `resize()` para asignar suficiente memoria para `nlocal` elementos para los vectores `local_a` y `local_b`. Defina `nlocal` como lo hicimos en clase. Puede asumir que el número de elementos N es un múltiplo del número de procesos `size` (i.e., no hay `rest`) (0.5%)
- En el proceso 0, asigne distintos valores a las entradas de los vectores `a` y `b`. Asigne estos valores de manera tal que $\mathbf{a} = \mathbf{b} = [1, 2, 3, \dots, N]$ (0.25%)
- Investigue y utilice la función `MPI_Scatter` para distribuir las secciones de los vectores `a` y `b` a sus respectivos procesos (0.5%)
- Calcule el producto interno de cada sección de tamaño `nlocal` en cada proceso (0.25%)
- Utilice `MPI_Reduce` para calcular el producto interno total con base en los productos internos de cada proceso (0.5%)
- Imprima el resultado usando el proceso 0 (0.25%)

No se preocupe por la escalabilidad. En general, este algoritmo permite aceleración que escala muy favorablemente, sin embargo, para observar estos resultados se deben estudiar vectores grandes. Si desea observar el beneficio de la aceleración del algoritmo, puede modificar el parámetro N para estudiar vectores grandes. Este ejercicio no es requerido para la tarea.